

Smart Task Manager

Name: Akshat Singh

Enrolment No.: 500120628

The Smart Task Manager is a React-based web application developed using AntiGravity and Claude Code through a multi-agent development workflow.

The application allows users to add tasks, mark tasks as completed, delete tasks, and view pending and completed tasks separately. Task data is stored using browser localStorage, ensuring persistence across page refreshes.

The project was developed using four AI agents:

1. Planning Agent – Defined requirements, architecture, workflow, and project structure.
2. Frontend Agent – Designed the responsive user interface and user experience.
3. Backend Logic Agent – Implemented task management functionality and localStorage persistence.
4. Integration & Testing Agent – Integrated all modules, tested functionality, and fixed issues.

Additional features include task search, priority management, due dates, and dark mode support.

This project demonstrates how agentic AI workflows can accelerate software development by enabling specialized agents to collaborate on planning, implementation, testing, and deployment.

(The code for this project is uploaded.)

Tools Used

Tool	Purpose
AntiGravity	Multi-agent orchestration
Claude Code	Code generation
React	Frontend framework

Tool	Purpose
JavaScript	Application logic
CSS	Styling
Local Storage	Data persistence
GitHub	Version control

Prompt 1:

You are a team of four AI agents collaborating to build a complete Smart Task Manager web application.

PROJECT REQUIREMENTS

Build a modern Smart Task Manager application that allows users to:

1. Add tasks
2. Mark tasks as completed
3. Delete tasks
4. View pending tasks separately
5. View completed tasks separately
6. Display total task count
7. Store tasks using localStorage so data persists after refresh

BONUS FEATURES

- Task search
- Task priority (High, Medium, Low)

- Due date selection
- Dark mode toggle

TECH STACK

- React
- JavaScript
- HTML
- CSS
- Local Storage
- Responsive Design

MULTI-AGENT WORKFLOW

=====

AGENT 1 – PLANNING AGENT

=====

Responsibilities:

- Analyze requirements
- Create project architecture
- Define folder structure
- Define application workflow
- Create implementation roadmap
- Assign responsibilities to other agents

Output:

1. Project Overview
2. Feature List
3. User Flow
4. Folder Structure
5. Development Roadmap

=====

AGENT 2 – FRONTEND AGENT

=====

Responsibilities:

Design a modern and professional UI.

Required Components:

- Application Header
- Task Input Form
- Add Task Button
- Search Bar
- Priority Dropdown
- Due Date Picker
- Pending Tasks Section
- Completed Tasks Section
- Task Counter
- Dark Mode Toggle

Design Requirements:

- Clean modern interface
- Card-based layout
- Mobile responsive
- Tablet responsive
- Desktop responsive
- Good spacing and typography
- Smooth hover effects
- Professional color palette

Output:

1. React Components
2. HTML Structure
3. CSS Styling
4. Responsive Layout

=====

AGENT 3 – BACKEND LOGIC AGENT

=====

Responsibilities:

Implement all application logic.

Required Features:

- Add Task
- Delete Task
- Complete Task
- Task Counter
- Search Tasks
- Priority Management
- Due Date Handling
- Local Storage Persistence

Requirements:

- Load saved tasks on startup
- Update localStorage automatically
- Use React Hooks
- Efficient state management
- Clean reusable functions

Output:

1. State Management Logic
2. Event Handlers
3. Local Storage Functions
4. Utility Functions

=====

AGENT 4 – INTEGRATION & TESTING AGENT

=====

Responsibilities:

Integrate frontend and backend.

Tasks:

- Connect UI with functionality
- Verify Add Task
- Verify Delete Task
- Verify Complete Task
- Verify Search
- Verify Priority Filter
- Verify Due Date Feature
- Verify Dark Mode
- Verify localStorage
- Verify Responsive Design

Fix:

- UI issues
- State issues
- Logic errors
- Performance issues

Output:

1. Final Integrated Code
2. Testing Report

3. Bug Fix Report
4. Deployment Instructions

FINAL DELIVERABLES

Generate:

1. Complete project structure
2. All React files
3. CSS files
4. Functional JavaScript code
5. README.md
6. Project explanation
7. Testing report
8. Setup instructions
9. Deployment instructions

The final application should be fully functional, modern, responsive, production-ready, and demonstrate effective collaboration between all four agents.

Prompt 2:

Approved.

Proceed with implementation and generate the complete Smart Task Manager application.

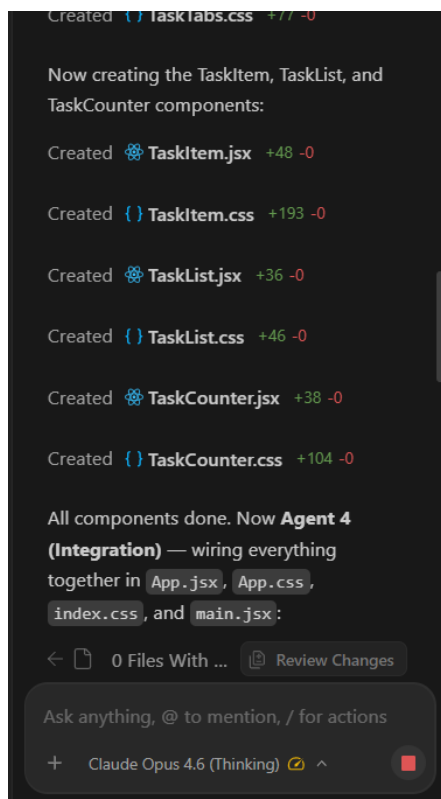
Requirements:

- * Generate all React components
- * Generate all CSS files

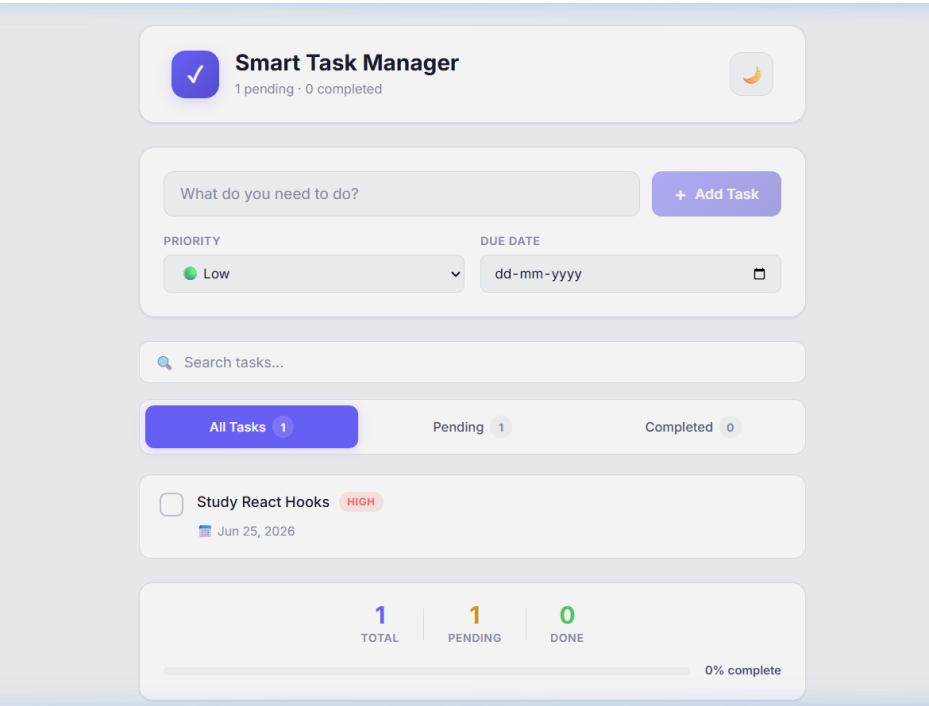
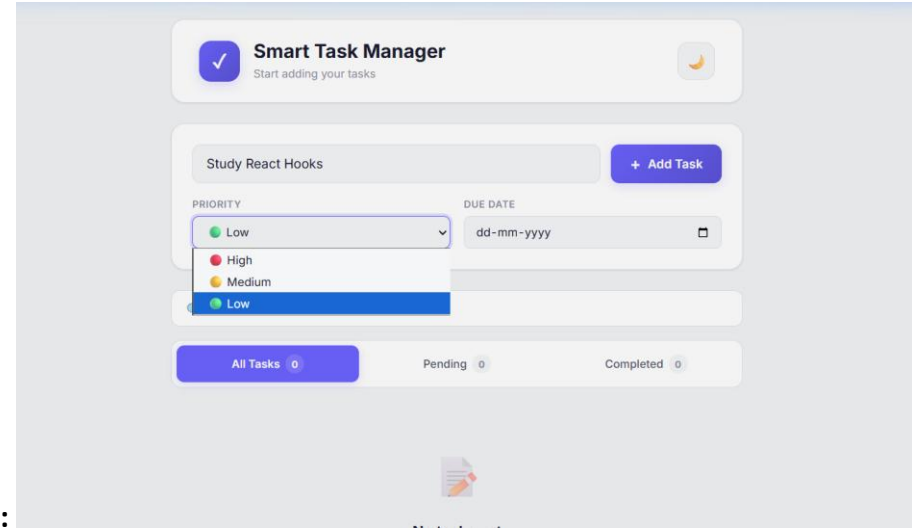
- * Generate all JavaScript logic
- * Implement localStorage persistence
- * Implement task add, delete, and complete functionality
- * Implement task counter
- * Separate pending and completed task sections
- * Include task search, priority levels, due dates, and dark mode
- * Create a responsive design for mobile, tablet, and desktop
- * Generate the complete project folder structure
- * Generate all files with full code content
- * Run integration testing and fix any issues automatically

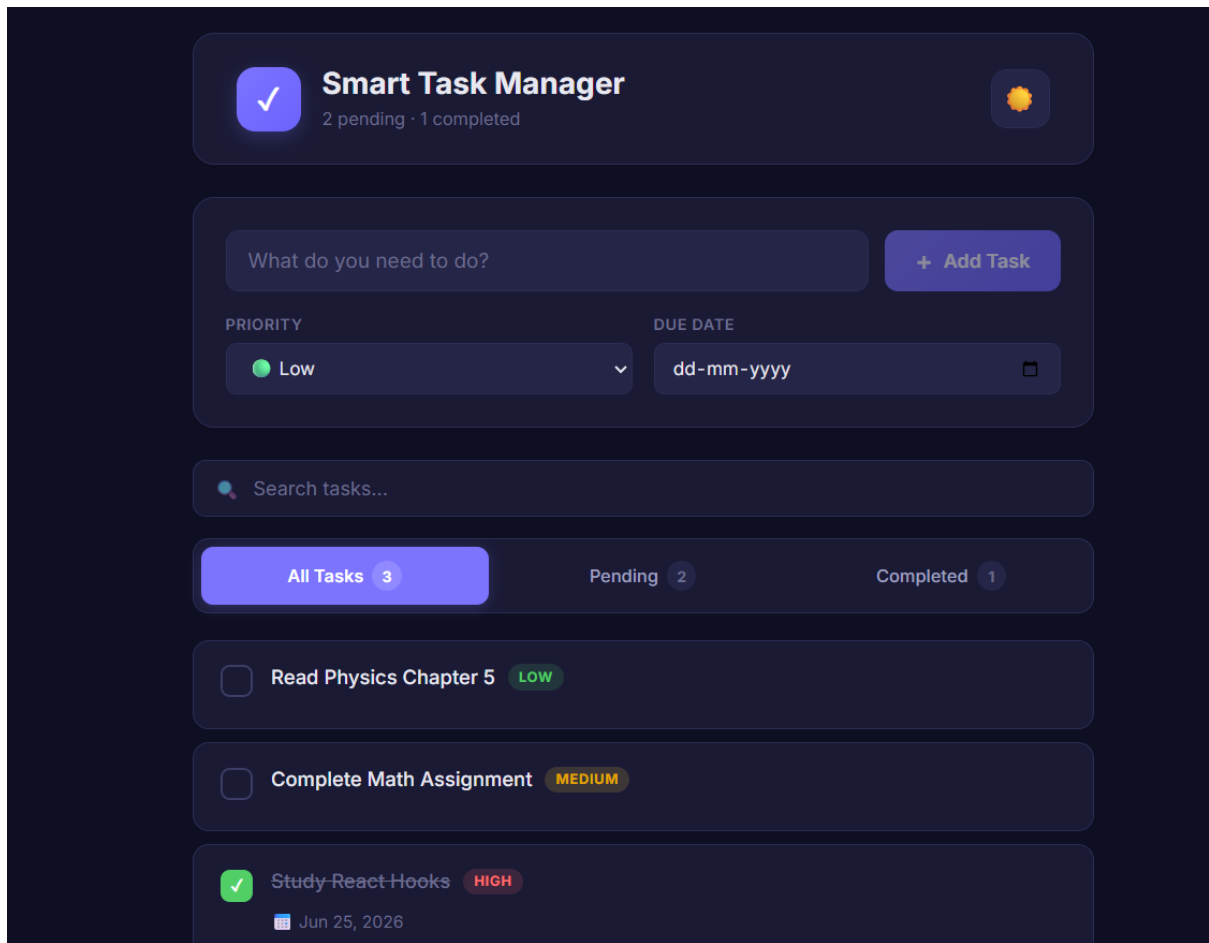
Do not stop after planning. Continue until the full application is built and all source code files are generated.

Agent Working:

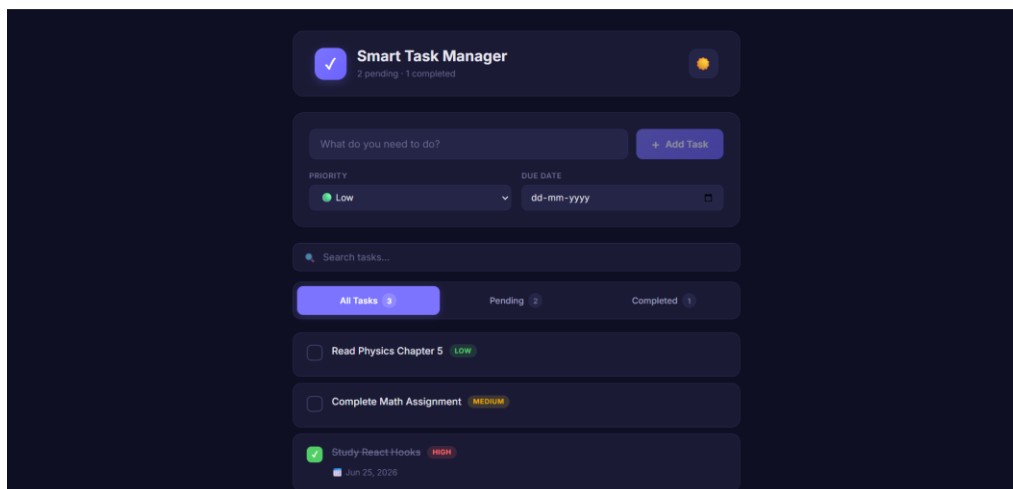


Agent Testing

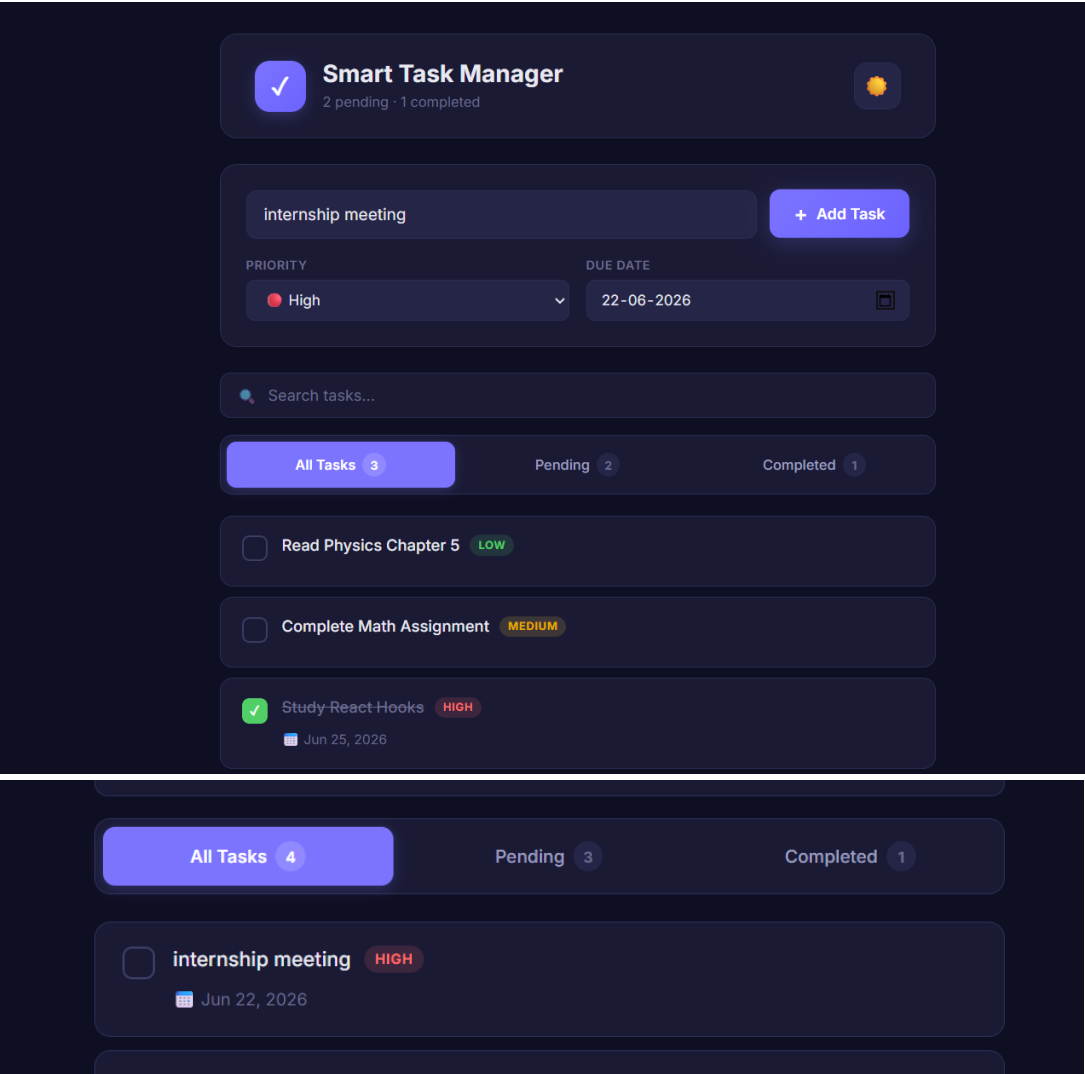




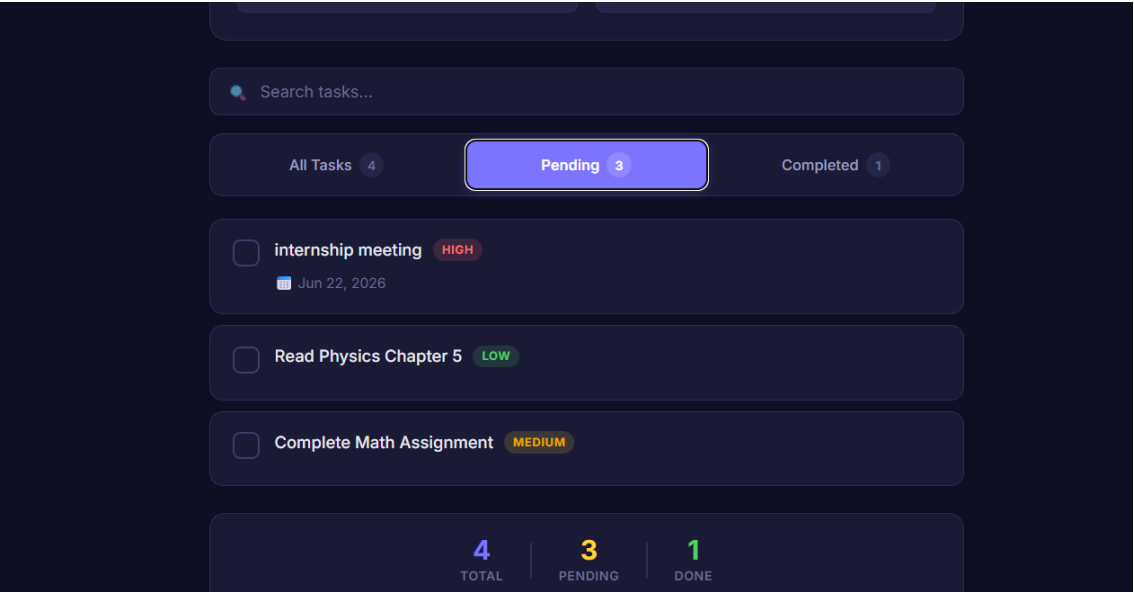
Final Working Application: Home



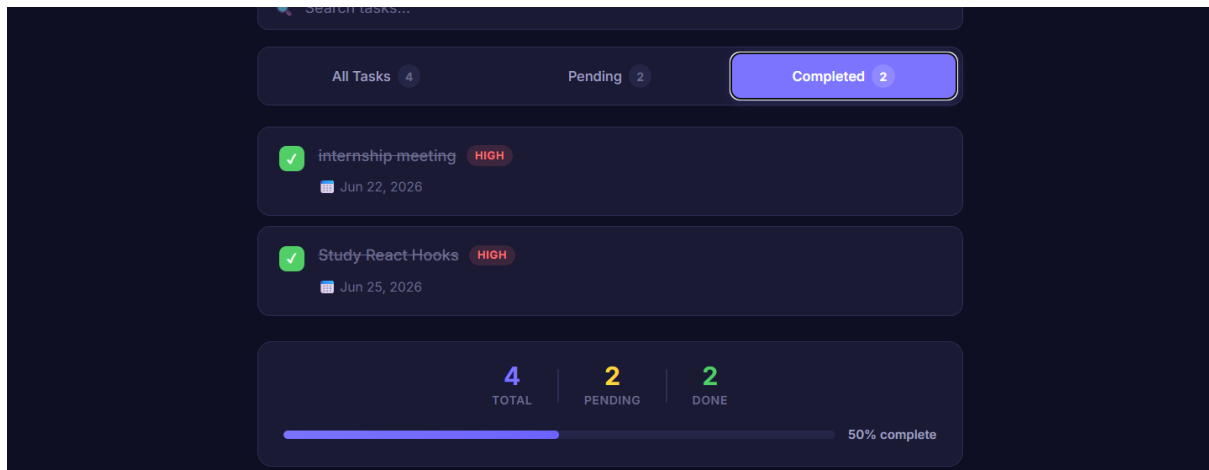
Add Task



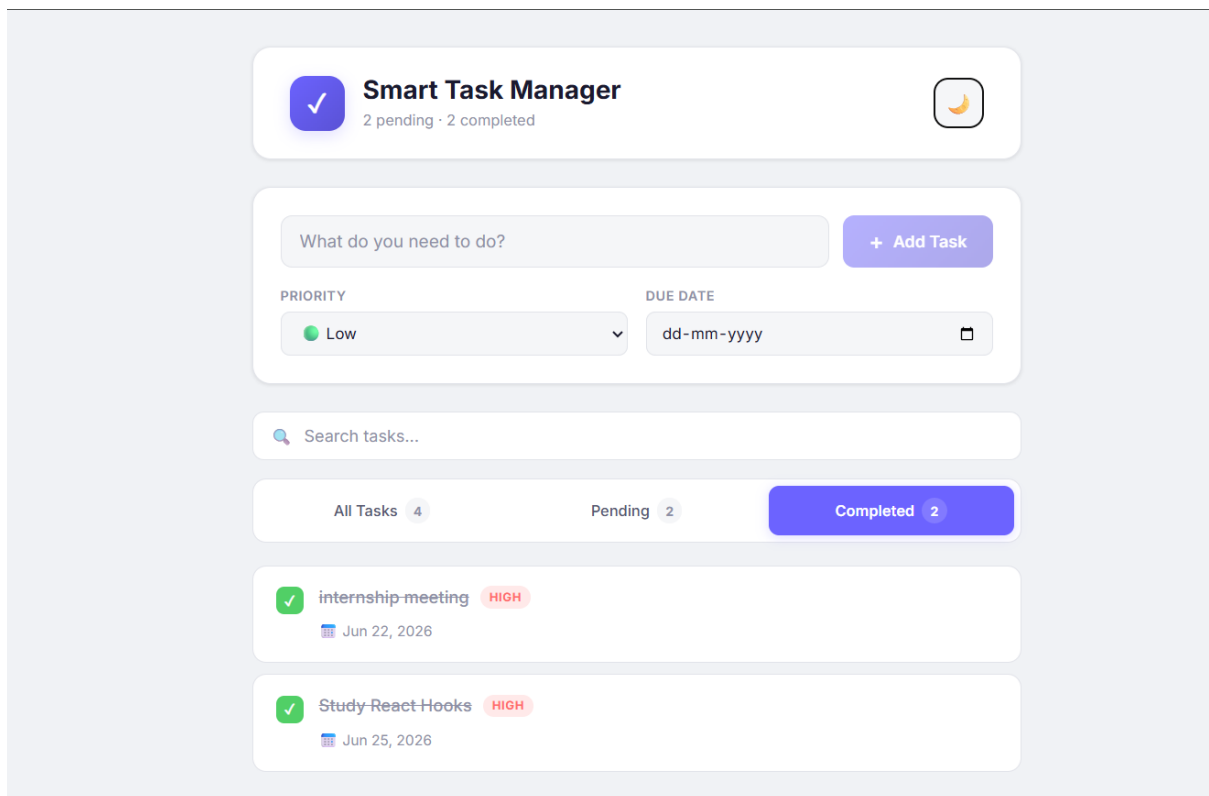
Pending tasks:



Completed Tasks:



Bright Mode:



Agent Output Summary

Planning Agent

- Designed project architecture
- Defined folder structure
- Prepared implementation roadmap

- Established development workflow

Frontend Agent

- Created responsive React user interface
- Implemented Header, TaskForm, SearchBar, TaskTabs, TaskList, TaskItem, and TaskCounter components
- Added light and dark mode support
- Implemented mobile, tablet, and desktop responsive layouts

Backend Logic Agent

- Developed task management functionality
- Implemented Add, Delete, and Complete task operations
- Added localStorage persistence
- Created custom hooks: useLocalStorage and useTasks
- Implemented search, priority filtering, and due date handling

Integration & Testing Agent

- Integrated all frontend and backend modules
- Verified all required features
- Executed production build successfully
- Confirmed zero build errors
- Generated deployment instructions and documentation

Conclusion: The Smart Task Manager was successfully developed using AntiGravity and Claude Code through a multi-agent workflow. The project demonstrates how AI agents can collaborate to rapidly design, develop, integrate, and test a complete web application with minimal manual coding.